

Inigo Quilez :: articles :: avoiding trigonometry - 2013



Intro

I think we should use less trigonometry in computer graphics. As my understanding of projections, reflections, and vector operations improved over time, I experienced a growing unease every time I saw trigonometry at the core of 3D algorithms inside rendering engines. These days I'm at a point where if I see some `asin`, `acos`, `atan`, `sin`, `cos` or `tan` in the middle of a 3D algorithm, I'll assume the code was written by an inexperienced programmer and that it needs review.

Now, don't get me wrong. Trigonometry **is** convenient and necessary for data input and for feeding the larger algorithm. What's wrong is when angles and trigonometry suddenly emerge deep in the internals of a 3D engine or algorithm out of nowhere. In other words, where the inputs and the outputs to an algorithm of function are vectors and geometry, and suddenly in the middle of the implementation trigonometry appears. Because, most of the time (if not all of the time) such a thing is unnecessarily complicated, error prone and overall unnecessary. It also gets in the way of elegance and truth, if that matters to you. Let me explain.

I've already discussed in other places before how the dot and cross products encode all the information you need to deal with orientation related operations. These two products respectively capture all that you've learnt about cosine and sine of angles. Actually, cosine and sine are really dot and cross products in disguise, which has been taken out of a geometric context and ripped of physicality. For example, they operate on "angles" which are a rather abstract thing compared to the actual "vectors" or "lines" that the two products operate on. Anyways, my claim for now is that when working with objects and vectors, going to angular and trigonometry land is an error. But let me make this claim more concrete with one typical example:

The suboptimal (wrong?) way to orientate an object

Say you have this function, which computes a matrix that rotates vectors around a given axis v by an amount of a radians. Many 3D engines, renderers or math libraries will have one such routine. The function will look something like this:

```

mat3x3 rotationAxisAngle( const vec3 & v, float a )
{
    const float si = sinf( a );
    const float co = cosf( a );
    const float ic = 1.0f - co;

    return mat3x3( v.x*v.x*ic + co,      v.y*v.x*ic - si*v.z,      v.z*v.x*ic + si*v.z,
                  v.x*v.y*ic + si*v.z,      v.y*v.y*ic + co,      v.z*v.y*ic - si*v.z,
                  v.x*v.z*ic - si*v.y,      v.y*v.z*ic + si*v.x,      v.z*v.z*ic + co );
}

```

So now imagine you are somewhere in the internals of your project, writing some code that needs to set the orientation of an object in a given direction. For example, you are aligning a spaceship to an animation path, by making sure the spaceship's **z** axis aligns with the path's tangent or direction vector **d**. So, you decide you want to use the `rotationAxisAngle()` function for that. Cool.

Then, you measure the angle between the spaceship's **z** axis and the desired orientation vector **d** so you know by how much you'll need to rotate it. Since you are a graphics programmer, you know you can do this by taking the dot product of the two vectors and then extracting the angle from it with an `acos()` call. Great. Also, because you know that `acos()` can return nonsense if its argument lays outside of the -1..1 range, you decide to clamp the dot product to -1..1 before calling `acos()`, just in case. "Because floating point". Fine. Although at this stage one kitten has already been murdered. But since you don't know that yet, you proceed to writing the rest of the code.

So, next you compute the rotation axis, which you know is the cross product of your **z** and **d** vectors, for all points in the spaceship need to rotate in planes parallel to that spanned by these two vectors. Then you decide to normalize the rotation axis vector, just because you often do so just to make sure things will work, regardless of whether you actually needed to normalize the vector or not. And so, in the end, your code looks like this:

```

const vec3  axi = normalize( cross( z, d ) );
const float ang = acosf( clamp( dot( z, d ), -1.0f, 1.0f ) );
const mat3x3 rot = rotationAxisAngle( axi, ang );

```

This does work. But I claim it's wrong. To see why, let's expand the code of `rotationAxisAngle()` in place and see the whole picture of what's mathematically going on:

```

const vec3  axi = normalize( cross( z, d ) );
const float ang = acosf( clamp( dot( z, d ), -1.0f, 1.0f ) );
const float co = cosf( ang );
const float si = sinf( ang );
const float ic = 1.0f - co;

```

```
const mat3x3 rot = mat3x3( axi.x*axi.x*ic + co,      axi.y*axi.x*ic - si*axi.z,
                           axi.x*axi.y*ic + si*axi.z,  axi.y*axi.y*ic + co,
                           axi.x*axi.z*ic - si*axi.y,  axi.y*axi.z*ic + si*axi.x );
```

As you have already probably noticed, we are performing an rather imprecise and expensive `acos()` call just to undo it immediately after by computing a `cos()` on its return value. `cos(acos(x))` is just `x` after all. This begs the question: why not skip the `acos()/cos()` chain altogether?

The key insight here is that this is not just a mere optimization (skipping `cos`, `acos` and `clamp` calls), but that maybe what we are revealing here is a deeper mathematical structure or relationship. Indeed, while sometimes simplifications and symmetries in mathematical expressions are certainly just accidental and meaningless, more often than not they actually are the consequence of deep truths. So let's keep exploring what's going on in our code, see if we can learn anything here.

So, a first roadblock seems to be that despite we could in principle simplify the `cos(acos(x))` chain, it seems we need to compute the angle or rotation anyways for our `sin()` function that comes right after the cosine. So we might not be able to simplify things much after all. However this is where we start using the dot and cross products.

If you are familiar with the cross product, you might know that it somehow encodes sines, just like dot products encode cosines. If you never realized this, and that's fine, that's why we are here today. A hint is that the length of a cross product is the length of the two vectors involved times the **sine** of the angle they form (while the dot product is the lengths of the vectors times the **cosine** of the angle they form).

So this is the point - wherever there's a dot product, chances are there's a cross product lying around, sometimes a bit hidden perhaps, that completes the picture of what's geometrically happening (an alignment and rotation in our case). And so, generally, for any parallel thing you can measure (dot products, cosines) there's a perpendicular component that you can measure too (cross product, sines). So, this is all a bit abstract perhaps, so let's go ahead and check this out:

The proper way to do it

We can extract the sine of the angle between `z` and `d` by just looking at the length of their cross product. Indeed, `z` and `d` are normalized so the length of their cross product is the sine of the angle they form times one and times one again! Which means we can and should rewrite our operation this way:

```
mat3x3 rotationAlign( const vec3 & d, const vec3 & z )
{
```

```

const vec3 ax = cross( z, d );
const float co = dot( z, d );
const float si = length( ax );
const vec3 v = ax/si;
const float ic = 1.0f - co;

return mat3x3( v.x*v.x*ic + co,      v.y*v.x*ic - si*v.z,      v.z*v.x*ic + si*v.z,
               v.x*v.y*ic + si*v.z,  v.y*v.y*ic + co,          v.z*v.y*ic - si*v.x,
               v.x*v.z*ic - si*v.y,   v.y*v.z*ic + si*v.x,      v.z*v.z*ic + co
            )
}

```

Good, no `normalize()`, no `acos()`, no `clamp()` anymore. We are definitely getting somewhere now. Let's keep going and also get rid of the length/square root operation by noticing that $si^2=1-co^2$ (since sine and cosine lay on the unit circle) and by propagating the $1/si$ term into the matrix and getting some cancellations. What we get is:

```

mat3x3 rotationAlign( const vec3 & d, const vec3 & z )
{
    const vec3 v = cross( z, d );
    const float c = dot( z, d );
    const float k = (1.0f-c)/(1.0f-c*c);

    return mat3x3( v.x*v.x*k + c,      v.y*v.x*k - v.z,      v.z*v.x*k + v.y,
                  v.x*v.y*k + v.z,      v.y*v.y*k + c,          v.z*v.y*k - v.x,
                  v.x*v.z*k - v.y,      v.y*v.z*k + v.x,      v.z*v.z*k + c      );
}

```

Lastly, we can simplify k to $k = 1/(1+c)$ which is more elegant and also moves the two singularities in k and whole function (d and z are parallel) into a single singularity (when d and z are the same vector, in which case there's no clear rotation that is best anyways (thanks Zoltan Vrana for this tip). So, the final function looks like this:

```

mat3x3 rotationAlign( const vec3 & d, const vec3 & z )
{
    const vec3 v = cross( z, d );
    const float c = dot( z, d );
    const float k = 1.0f/(1.0f+c);

    return mat3x3( v.x*v.x*k + c,      v.y*v.x*k - v.z,      v.z*v.x*k + v.y,
                  v.x*v.y*k + v.z,      v.y*v.y*k + c,          v.z*v.y*k - v.x,
                  v.x*v.z*k - v.y,      v.y*v.z*k + v.x,      v.z*v.z*k + c      );
}

```

Conclusion

So look at that beauty - there aren't any trigonometric or inverse trigonometric functions, nor clamp or normalizations or even square roots. This is great from a stability and performance point of view. And also, we have solved a problem relating vectors using only vectors, which just feels right.

Now, I'm not going to claim that in this particular example of today we got an intuitive interpretation of the final matrix. Neither we had it in the beginning, so nothing was lost there. Indeed, all I can think of doing to the matrix above is decompose it as

$$M = \frac{\mathbf{v} \cdot \mathbf{v}^T}{1+c} + \begin{pmatrix} c & -\mathbf{v} \cdot \mathbf{z} & \mathbf{v} \cdot \mathbf{y} \\ \mathbf{v} \cdot \mathbf{z} & c & -\mathbf{v} \cdot \mathbf{x} \\ -\mathbf{v} \cdot \mathbf{y} & \mathbf{v} \cdot \mathbf{x} & c \end{pmatrix}$$

The point is, in almost all situations you can perform similar trigonometric simplifications and slowly untangle the math to uncover a simpler vector expression that describes the same problem. Also, oftentimes you can actually rethink the whole problem in terms of vectors from the beginning, which is more intuitive anyways. For that you'll need to develop some intuition about dot products, cross products and projections. But once you have it you'll find that many of the trigonometric identities you find in literature are just a statement about some particular vector configurations, so you'll never have to memorize or look up the identities again. But I digress.

I'll just conclude saying that part of the problem with overusing trigonometry in 3D rendering codebases comes from poorly designed third party APIs, like that of `rotationAxisAngle(v,a)` or WebXR stereo projection information, which work on angles and force their users to use angles too, infecting our codebases. Sometimes the cost is a performance or stability hit; some other times there's no performance cost but either way I think we should strive to doing more vectors operations and less trigonometry!

inigo quilez - learning computer graphics since 1994